

FUNDAMENTOS DE PRORGRAMAÇÃO

Operadores



Teach
3035

Índice

Operadores.....	03
Operadores Aritméticos.....	04
Operadores de Atribuição.....	05
Operadores de Comparação.....	07
Operadores Lógicos.....	09
Operadores de Incremento e Decremento.....	10
Operadores Condicionais.....	11
Operador de Concatenação.....	13

Operadores



O JS possui uma variedade de **operadores** que permitem manipular e realizar operações em valores. Os operadores em JavaScript podem ser classificados em diferentes categorias:

1. Operadores Aritméticos
2. Operadores de Atribuição
3. Operadores de Comparação
4. Operadores Lógicos
5. Operadores de Incremento e Decremento
6. Operadores Condicionais
7. Operador de Concatenação

Operadores Aritméticos



Os operadores aritméticos são usados para realizar operações matemáticas em valores numéricos.

- **Adição (+): Realiza a adição de dois valores.**

```
let resultado = 5 + 3; // resultado terá o valor 8
```

- **Subtração (-): Subtrai um valor de outro.**

```
let resultado = 10 - 4; // resultado terá o valor 6
```

- **Multiplicação (*): Multiplica dois valores.**

```
let resultado = 2 * 5; // resultado terá o valor 10
```

- **Divisão (/): Divide um valor pelo outro.**

```
let resultado = 20 / 5; // resultado terá o valor 4
```

- **Resto da divisão (%): Retorna o resto da divisão de um valor pelo outro.**

```
let resultado = 15 % 4; // resultado terá o valor 3
```

Operadores Aritméticos



- **Exponenciação (**):** cálculo de uma potência, onde um número é elevado a uma determinada potência

```
let resultado = 2 ** 3; // resultado terá o valor 8 (2 elevado a 3)
```

Operadores de Atribuição



É usado para atribuir um valor a uma variável ou a uma propriedade de um objeto. É representado pelo símbolo de igual (=). Exemplo:

```
let number = 5; // atribuindo o valor 5 para a variável
```

Existem também os operadores de atribuição composta que serão explicados abaixo:

- **Atribuição de Soma (+=):** Realiza uma adição entre o valor existente da variável e outro valor, e atribui o resultado à própria variável.

```
number += 3; // Equivalente a: number = number + 3;  
// Agora, o valor de number é 8
```

Operadores de Atribuição



- **Atribuição de Subtração (--):** Realiza uma subtração entre o valor existente da variável e outro valor, e atribui o resultado à própria variável.

```
number -= 3; // Equivalente a: number = number - 3;  
// Agora, o valor de number é 2
```

- **Atribuição de Multiplicação (*=):** Realiza uma multiplicação entre o valor existente da variável e outro valor, e atribui o resultado à própria variável.

```
number *= 2; // Equivalente a: number = number * 2;  
// Agora, o valor de number é 10
```

- **Atribuição de Divisão (/=) e Resto (%=):**

```
number /= 2; // Equivalente a: number = number/2;  
number %= 3; // Equivalente a: number = number%2;
```

Por fim, a atribuição composta de divisão e resto armazenará na variável `number` o resultado da divisão de `number` por 2, e o resto da divisão de `x` por 3, respectivamente.

Operadores de Comparação



São usados para comparar valores e retornar um resultado booleano (verdadeiro ou falso) com base na comparação.

- **Igual (==):** Verifica se dois valores são iguais, sem considerar o tipo.

```
5 == 5; // true  
'10' == 10; // true (coerção de tipo é realizada)
```

- **Estritamente Igual (===):** Verifica se dois valores são iguais, considerando o tipo.

```
5 === 5; // true  
'10' === 10; // false (tipos diferentes)
```

- **Diferente (!=):** Verifica se dois valores são diferentes, sem considerar o tipo.

```
5 != 3; // true  
'10' != 10; // false (coerção de tipo é realizada)
```

Operadores de Comparação



- **Estritamente Diferente (!==):** Verifica se dois valores são diferentes, considerando o tipo.

```
5 !== 3; // true  
'10' !== 10; // true (tipos diferentes)
```

- **Maior que (>):** Verifica se o valor à esquerda é maior que o valor à direita.

```
8 > 5; // true
```

- **Maior ou igual a (>=):** Verifica se o valor à esquerda é maior ou igual ao valor à direita.

```
5 >= 5; // true  
7 >= 10; // false
```

- **Menor que (<):** Verifica se o valor à esquerda é menor que o valor à direita.

```
3 < 5; // true
```

Operadores de Comparação



- **Menor ou igual a (<=):** Verifica se o valor à esquerda é menor ou igual ao valor à direita.

```
5 <= 5; // true  
8 <= 3; // false
```

Operadores Lógicos



Há três operadores lógicos principais: **AND (E)**, **OR (OU)** e **NOT (NÃO)**. Esses operadores são usados para combinar ou negar expressões lógicas e retornar um resultado booleano.

- **AND (&&):** Retorna verdadeiro se TODAS as expressões avaliadas forem verdadeiras.

```
true && true; // true  
true && false; // false
```

- **OU (||):** Retorna verdadeiro se **PELO MENOS UMA** das expressões avaliadas for verdadeira.

```
true || false; // true  
false || false; // false
```

Operadores Lógicos



- **NOT(!):** Inverte o valor de uma expressão, retornando o oposto do valor original.

```
!true; // false  
!false; // true
```

Operadores de Incremento e Decremento



- **Incremento (++):** O operador de incremento aumenta o valor da variável em uma unidade.

```
let number = 5;  
let resultado = ++number;  
// O valor de resultado será 6, e o valor de number também será 6
```

```
let number = 5;  
let resultado = number++;  
// O valor de resultado será 5, e o valor de number será 6
```

Operadores de Incremento e Decremento



- **Decremento (--):** O operador de decremento diminui o valor da variável em uma unidade.

Pré-decremento:

```
let number = 5;  
let resultado = --number;  
// O valor de resultado será 4, e o valor de number também será 4
```

Pós-decremento:

```
let number = 5;  
let resultado = number--;  
// O valor de resultado será 5, e o valor de number será 4
```

Operadores Condicionais



Permitem executar diferentes blocos de código com base em condições específicas.

- **Operador Condicional Ternário (?:):** Também conhecido como operador ternário, é uma forma concisa de escrever uma estrutura condicional if-else em uma única linha. A sintaxe do ternário é essa:

```
condição ? expressão1 : expressão2;
```

Operadores Condicionais



Se a condição for verdadeira, a expressão1 será executada; caso contrário, a expressão2 será executada.

Veja o exemplo abaixo:

```
let age = 18;  
let message = (age >= 18) ? 'Maior de idade' : 'Menor de idade';  
// O valor de mensagem será 'Maior de idade'
```

Além do ternário, podemos utilizar os operadores lógicos (&&, ||, !) para avaliar uma condição e executar um bloco de código com base no resultado.

Veja o exemplo abaixo:

```
let age = 25;  
let hasDriverLicense = true;  
if (age >= 18 && hasDriverLicense) {  
  console.log('Pode dirigir');  
} else {  
  console.log('Não pode dirigir');  
}  
// 'Pode dirigir'
```

Operador de Concatenação



É usado para combinar strings (cadeias de caracteres) em uma única string. Veja um exemplo abaixo:

```
let string1 = 'Olá';  
let string2 = 'mundo';  
let result = string1 + ' ' + string2;  
// O valor de result será 'Olá mundo'
```

É importante mencionar que o operador de concatenação também pode ser usado com outros tipos de dados, como números. No entanto, ele irá converter os valores não string em strings antes de realizar a concatenação.